

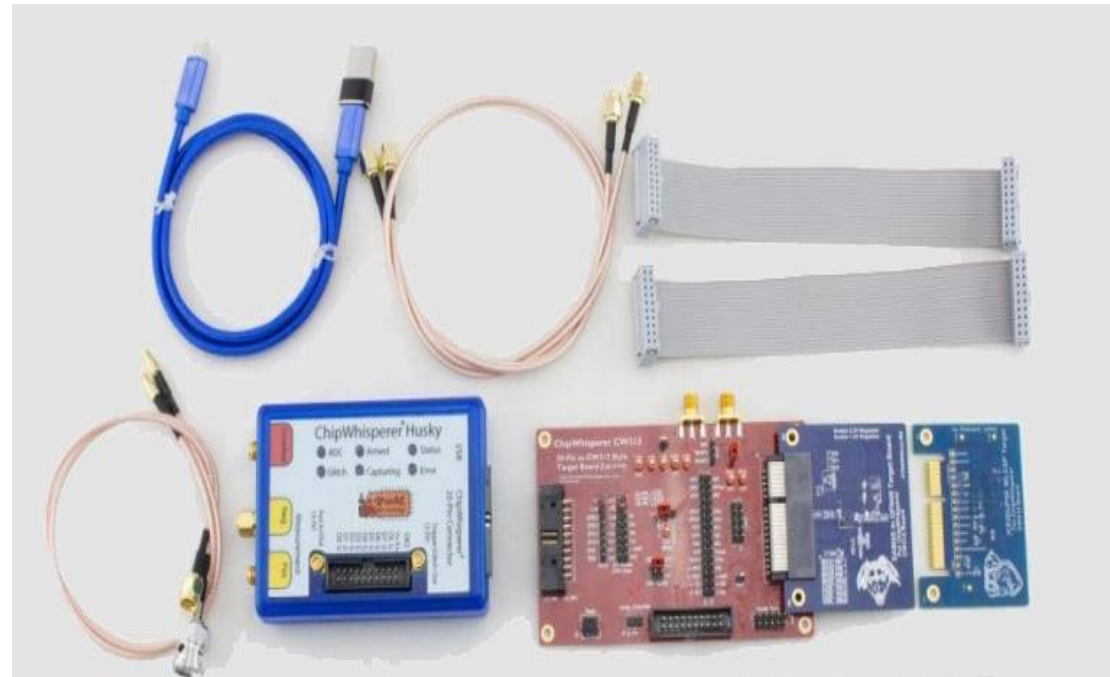
Side-channel power attack against a cryptographic software

Metehan Koç
Renaud Pacalet

PROJECT OVERVIEW

Objective: Capture power traces of ASCON algorithm to extract private key by side channel attacks.

Target Hardware: 32-bit ARM chip via CW-Husky.

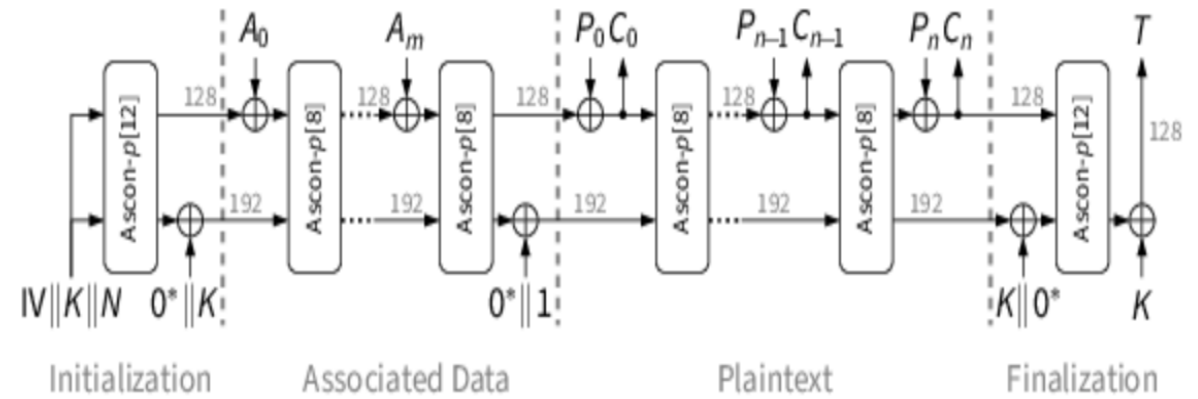


WHAT IS ASCON?

- **Purpose:** Provides high security with low overhead, making it ideal for the **Internet of Things (IoT)**, embedded sensors, and medical devices.
- **Functionality:** It is primarily an **Authenticated Encryption with Associated Data (AEAD)** scheme, meaning it provides both **confidentiality** (encryption) and **integrity** (tamper protection) in a single process.
- **Design Philosophy:** It is based on a **sponge construction**, similar to the SHA-3 (Keccak) hashing standard.

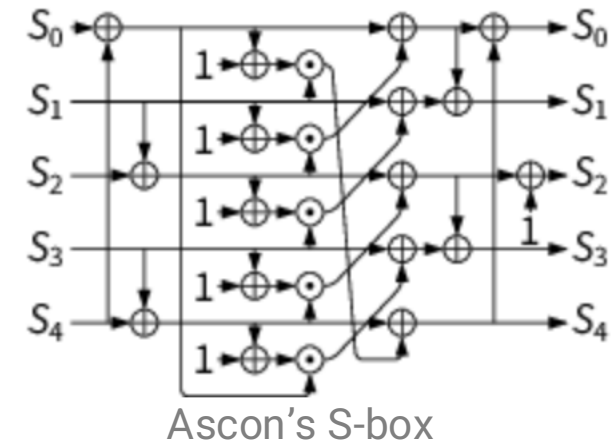
PHASES OF ASCON

- **Initialization:** Mixes Key, Nonce, and IV into the 320-bit state.
- **Associated Data:** Absorbs non-encrypted data into the state.
- **Processing:** XORs plaintext/ciphertext to encrypt or decrypt.
- **Finalization:** Generates the 128-bit tag for verification.



ASCON PERMUTATIONS

- **Addition of Round Constants:** XORs a unique 1-byte constant into the middle word (S_2).
- **Nonlinear Substitution Layer:** Applies a 5-bit S-box "vertically" across all 5 words (64 times in parallel).
- **Linear Diffusion Layer:** XORs each 64-bit word with two of its own rotated copies.



$$S_0 := S_0 \oplus (S_0 \ggg 19) \oplus (S_0 \ggg 28)$$

$$S_1 := S_1 \oplus (S_1 \ggg 61) \oplus (S_1 \ggg 39)$$

$$S_2 := S_2 \oplus (S_2 \ggg 1) \oplus (S_2 \ggg 6)$$

$$S_3 := S_3 \oplus (S_3 \ggg 10) \oplus (S_3 \ggg 17)$$

$$S_4 := S_4 \oplus (S_4 \ggg 7) \oplus (S_4 \ggg 41)$$

Ascon's linear layer

ASCON STATE LOADING

- S_0 : 64-bit IV (contains algorithm-specific constants).
- S_1, S_2 : 128-bit Secret Key (K).
- S_3, S_4 : 128-bit Nonce (N).

```
ascon_state_t s;  
s.x[0] = ASCON_128A_IV;  
s.x[1] = K0;  
s.x[2] = K1;  
s.x[3] = N0;  
s.x[4] = N1;
```

SUBSTITUTION LAYER

```
// --- Substitution Layer ---
s->x[0] ^= s->x[4];
s->x[4] ^= s->x[3];
s->x[2] ^= s->x[1];

t.x[0] = s->x[0] ^ (~s->x[1] & s->x[2]); // t.x[0] = (IV xor N1) xor (not K0 and K1 xor K0 xor RC)
t.x[1] = s->x[1] ^ (~s->x[2] & s->x[3]); // t.x[1] = K0 xor (not (K1 xor K0 xor RC)&N0)
t.x[2] = s->x[2] ^ (~s->x[3] & s->x[4]); // t.x[2] = (K1 xor K0 xor RC) xor (not N0 & (N1 xor N0))
t.x[3] = s->x[3] ^ (~s->x[4] & s->x[0]); // t.x[3] = N0 xor (not (N1 xor N0) and (IV xor N1))
t.x[4] = s->x[4] ^ (~s->x[0] & s->x[1]); // t.x[4] = (N1 xor N0) xor (not (IV xor N1) and K0) --> r2 = low, r3 = high

t.x[1] ^= t.x[0];
t.x[0] ^= t.x[4];
t.x[3] ^= t.x[2];
t.x[2] = ~t.x[2];
```

```
4002c6: ea29 0e03 bic.w lr, r9, r3 @ lr = s->x[1]_high & ~s->x[0]_high
4002ca: ea8e 0e01 eor.w lr, lr, r1 @ lr = lr ^ s->x[4]_high

4002e4: ea2a 0b08 bic.w fp, sl, r8 @ fp = s->x[1]_low & ~s->x[0]_low
4002f0: ea8b 0b06 eor.w fp, fp, r6 @ fp = fp ^ s->x[4]_low
```

Assembly
presentation of
C code

IMPLEMENTATION AND KAT VERIFICATION

- **Implementation:** ASCON AEAD encryption and decryption algorithms implemented by using C (ascon.c, ascon.h).
- **Test Dataset:** Utilized a comprehensive suite of **1,000 test cases** derived from the official reference implementation.
- **Software Validation:** Developed a C script (test_ascon.c) to verify encryption/decryption consistency across the entire dataset.
- **Result:** ASCON implementation pass the all tests and it verified the correctness of the implementations.

FIRMWARE

- Target firmware is implemented in C.
- Removed get_AD and get_PT to optimize firmware for collecting traces.
- **get_key**: Sets the private key for ASCON.
- **get_nonce**: Sets the nonce and starts the first permutation round.

```
uint8_t get_key(uint8_t* k, uint8_t len)
{
    saved_K0 = load_word(k, 8);
    saved_K1 = load_word(k + 8, 8);

    base_state.x[0] = ASCON_128A_IV;
    base_state.x[1] = saved_K0;
    base_state.x[2] = saved_K1;

    return 0x00;
}

uint8_t get_nonce(uint8_t* n, uint8_t len) {
    // Copy the pre-calculated base state
    ascon_state_t s = base_state;

    // Load the fresh nonce
    s.x[3] = load_word(n, 8);
    s.x[4] = load_word(n + 8, 8);

    trigger_high();

    // Running 12 rounds is 12x slower and adds no value to the leakage of Round 1.
    ascon_round(&s, ASCON_RC[0]);

    trigger_low();

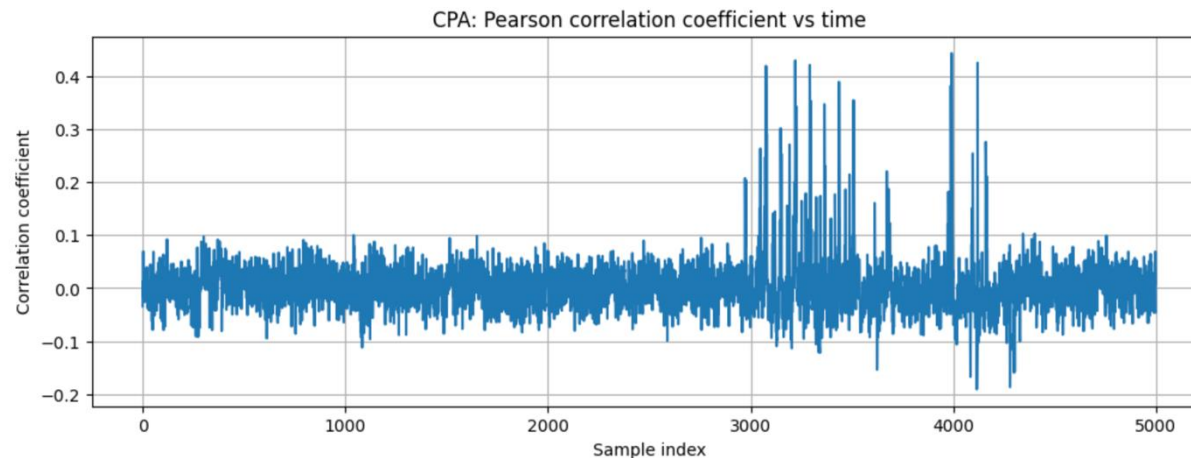
    return 0x00;
}
```

SIDE CHANNEL ATTACK TO ASCON

- To handle the complexity of the 32-bit K_0_low register, the attack uses a **Divide-and-Conquer** strategy combined with an iterative power model.
- **Target:** K_0_low (Lower 32 bits of the first 64-bit key chunk).
- **Strategy:** Two 16-bit sub-key guesses (65,536 hypotheses each) to reduce computational overhead.
- **Power Model:** Hamming Weight (HW) of the intermediate state R_8 , modeled as:
$$H = HW(K_{guess} \& (\sim (IV_{low} \oplus Nonce_{low})))$$
- **Dataset:** 500,000 power traces processed using vectorized NumPy operations for high-speed correlation calculation.

SYNCHRONIZATION OF TRACES

- Before running a full-scale attack on an unknown key, we use a **Known Key** to verify that the power traces are correctly synchronized with the ASCON cryptographic operations.
- **Hypothesis:** If our model (R_8 register state) is correct and the traces are aligned, we should see a distinct correlation spike at the exact moment that register is manipulated in hardware.



EXECUTION PIPELINE

- **Trace Centering:** Pre-calculates mean traces and standard deviation to optimize the Pearson correlation loop.
- **Iterative Recovery:**
 - **Phase 1:** Attacks Bytes 0-1 to find the first 16 bits.
 - **Phase 2:** Attacks Bytes 2-3 while maintaining the state of the previously recovered bits.
- **Statistical Selection:** The candidate with the highest absolute Pearson correlation coefficient is selected as the winner for each segment.

RESULTS FOR KO_LOW

- K0 recovered successfully (0x16157e2b)
- Top results are all close to the actual key

Top 10 Candidates for Bytes 0-1:

Rank	Key Guess	Correlation
------	-----------	-------------

1	0x7e2b	0.520455
2	0x7e2a	0.504724
3	0x7e29	0.503822
4	0x6e2b	0.503502
5	0xeec6	0.502487
6	0x7e23	0.499363
7	0x3e2b	0.497228
8	0x30f7	0.496682
9	0x7e0b	0.492916
10	0x762b	0.489850

=====
Attacking Bytes 2 and 3 (16-bit guess)
=====

Computing 65536 hypotheses...

Computing correlations...

Current recovered state: 0x16157e2b

Top 10 Candidates for Bytes 2-3:

Rank	Key Guess	Correlation
------	-----------	-------------

1	0x1615	0.652251
2	0x1215	0.637004
3	0x1614	0.635163
4	0x1611	0.634064
5	0x1415	0.630481
6	0x1605	0.628696
7	0x0615	0.627587
8	0x3615	0.621592
9	0x1655	0.620459
10	0x1214	0.619693

RESULTS FOR KO_HIGH

- Results for all the bytes are wrong.
- Correct bytes are not in top 10 guesses.
- Possible reasons are false implementation, initial value of the register, noises.

Top 10 Candidates for High Bytes 0-1:

Rank	Key Guess	Correlation
1	0x0000	0.857971
2	0x0100	0.846663
3	0x0001	0.846483
4	0x0040	0.846234
5	0x0020	0.845785
6	0x0004	0.845236
7	0x0080	0.844413
8	0x0002	0.844195
9	0x0800	0.844165
10	0x0008	0.844094

=====
Attacking Bytes 6 and 7 (High Word)
=====

Computing 65536 hypotheses...
Computing correlations...

Top 10 Candidates for High Bytes 2-3:

Rank	Key Guess	Correlation
1	0x0000	0.857971
2	0x0020	0.846953
3	0x4000	0.846805
4	0x0040	0.846352
5	0x0400	0.846175
6	0x0100	0.845626
7	0x0008	0.845381
8	0x1000	0.845198
9	0x0004	0.845099
10	0x0002	0.844890

SUMMARY

1. Implementation & Environment

- **Target:** Successful implementation of **Ascon** authenticated encryption on firmware.
- **Data Acquisition:** Power traces collected from the target hardware.

2. Testing & Synchronization

- **Testing:** Ascon implementation tested both on local and target
- **Trace Alignment:** Verified synchronization by comparing trace graphs across different cryptographic keys

3. Side-Channel Attack (SCA)

- Side channel attack is performed by using Pearson Correlation
- **K_{0_low}:** Successfully recovered with high confidence
- **K_{0_high}:** Currently not recovered; further noise reduction or trace amplification required.

REFERENCES

- <https://ascon.isec.tugraz.at/index.html>
- <https://github.com/newaetech/chipwhisperer>
- <https://chipwhisperer.readthedocs.io/en/latest/index.html>
- <https://github.com/ascon/ascon-c/>

Thank you for listening